

Progression — NSI Terminale

Année 2026-2027 · Zone C (Créteil) · Gantt généré automatiquement

P1 · 1 sept. → 16 oct. · 7 semaines

D1 S1 · Structures de données linéaires Cr · De · Ra

- Rappels : listes, tuples, dictionnaires, p-uplets nommés
- Type abstrait vs implémentation
- Piles (LIFO) : interface et implémentations
- Files (FIFO) : interface et implémentations
- Listes chaînées (notion)
- Applications (historique, file d'attente, parcours)
- 🎯 STR1 — Spécifier une structure de données par son interface ; distinguer interface et implémentation ; écrire plusieurs implémentations d'une même structure (ex. file avec tableau ou deux piles).
- 🎯 STR2 — Distinguer des structures linéaires (listes, piles LIFO, files FIFO, dictionnaires) par le jeu de leurs méthodes ; choisir une structure adaptée à la situation ; distinguer recherche dans une liste et dans un dictionnaire.
- ✂ AUTO-DICT1 — Compléter un accès sécurisé à un dictionnaire avec .get()
- 🚀 TP-STR-01 — File d'événements dans un jeu (1h30, ★★★)
- 🚀 TP-STR-02 — Pile d'historique / undo (1h30, ★★★)
- 🚀 TP-STR-03 — Gestion de projectiles et ennemis avec listes (2h, ★★★)

B1 S2 · Bases de données Cr · De · Ra · Ce

- Modèle relationnel : relations, attributs, domaines
- Clés primaires et étrangères ; schéma relationnel
- Cohérence et contraintes d'intégrité
- SQL : SELECT, WHERE, ORDER BY, jointures
- SQL : INSERT, UPDATE, DELETE
- Agrégation (COUNT, GROUP BY)
- 🎯 BDD1 — Identifier les concepts définissant le modèle relationnel (relation, attribut, domaine, clef primaire, clef étrangère, schéma relationnel).
- 🎯 BDD4 — Identifier les composants d'une requête ; construire des requêtes d'interrogation (SELECT, FROM, WHERE, JOIN) et de mise à jour (UPDATE, INSERT, DELETE) en SQL.
- 🎯 BDD2 — Distinguer la structure d'une base de données de son contenu ; repérer des anomalies dans un schéma (redondances, anomalies d'insertion/suppression/mise à jour).
- 🎯 BDD3 — Identifier les services rendus par un SGBD relationnel : persistance, gestion des accès concurrents, efficacité des requêtes, sécurisation des accès.
- ✂ AUTO-SQL1 — Compléter le squelette d'une requête SQL de sélection filtrée
- ✂ AUTO-SQL2 — Connaître la clause SQL de tri
- 🚀 TP-BDD-01 — Leaderboard SQLite d'un jeu (2h, ★★★)
- 🚀 TP-BDD-02 — Base de données de cartes TCG (2h, ★★★)
- 🚀 TP-BDD-03 — Requêtes SQL sur personnages, raretés, scores, équipes (1h30, ★★★)

B2 Éval ECE #1 — SQL (BNS 2026) Cr · Ra

- Évaluation individuelle chronométrée (ex. 1h), sujet unique choisi selon le niveau de chaque élève.
- Sujet 15 — cabinet vétérinaire (SQL + sqlite3, requêtes SELECT/JOIN, validation de format) : https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data2026/26_BCG_NSI_15/sujet.pdf
- (Seul sujet 100% SQL de la BNS 2026 — bon point d'ancrage juste après S2.)

o1 S3 · Programmation orientée objet Cr · De · Ra

- Classes, objets, attributs, méthodes
- Constructeur, encapsulation
- Interface vs implémentation
- Réécrire piles / files en POO
- Vocabulaire (instance, self)
- 🎯 POO1 — Écrire la définition d'une classe ; accéder aux attributs et méthodes d'une classe (hors polymorphisme et héritage).
- ✂ AUTO-POO1 — Compléter une définition de classe (self, constructeur)
- ✂ AUTO-POO2 — Corriger un oubli de self. sur un attribut
- 🚀 TP-POO-01 — Classes Player, Enemy, Projectile (2h, ★★★)
- 🚀 TP-POO-02 — Simulateur de combat Pokémon / RPG (2h30, ★★★★★)

— Vacances de la Toussaint —

P2 · 2 nov. → 18 déc. · 7 semaines

o2 Éval ECE #2 — dictionnaires & POO simple Cr · Ra

- Évaluation individuelle chronométrée, sujet unique adapté au niveau.
- Sujet 2 — salaires en entreprise (dico, moyenne conditionnelle, erreur de type) :
https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data/EP_sujets0_2/EP-BAC-NSI-Sujet2.pdf
- Sujet 4 — croissance de plantes (dico, bug classique de suppression pendant un parcours de liste) :
https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data2026/26_BCG_NSI_4/sujet.pdf
- Sujet 6 — boutique de smoothies (POO simple, self, méthodes) :
https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data2026/26_BCG_NSI_6/sujet.pdf
- Sujet 17 — budget associatif (dico, agrégation, erreur de range) :
https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data2026/26_BCG_NSI_17/sujet.pdf

P1 S4 · Langages & programmation Cr · Ra · De · Ce

- Récursivité : principe, pile d'appels, exemples
- Modularité : fonctions, modules, import
- Spécification, jeux de tests, assertions
- Mise au point : débogage, invariants
- Notion de complexité (coût d'un algorithme)
- Paradigmes (impératif, fonctionnel) — sensibilisation
- 🎯 LAN4 — Distinguer sur des exemples les paradigmes impératif, fonctionnel et objet ; choisir le paradigme selon le champ d'application.
- 🚀 TP-REC-01 — Génération récursive de donjon (2h30, ★★★★★)
- 🚀 TP-REC-02 — Exploration récursive d'une grille (1h30, ★★★)
- 🚀 TP-REC-03 — Arbre de dialogues de PNJ (2h, ★★★)

P2 Éval ECE #3 — récursivité & structures imbriquées Cr · Ra

- Évaluation individuelle chronométrée, sujet unique adapté au niveau.
- Sujet 5 — empreinte carbone (dictionnaire récursif imbriqué, fonction récursive incomplète) : https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data2026/26_BCG_NSI_5/sujet.pdf
- Sujet 11 — prédiction d'habitat (distance euclidienne, k plus proches voisins) : https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data2026/26_BCG_NSI_11/sujet.pdf
- Sujet 19 — gestion de réservoirs d'eau (dico, vulnérabilité par district) : https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data2026/26_BCG_NSI_19/sujet.pdf
- 🎯 LAN2 — Écrire un programme récursif ; analyser le fonctionnement d'un programme récursif.
- 🎯 LAN1 — Comprendre que tout programme est aussi une donnée et que la calculabilité ne dépend pas du langage ; montrer sans formalisme que le problème de l'arrêt est indécidable.
- 🎯 LAN3 — Utiliser des API ou des bibliothèques en exploitant leur documentation ; créer des modules simples et les documenter.
- 🎯 LAN5 — Répondre aux causes typiques de bugs (typage, effets de bord, débordements de tableaux, conditionnelles non exhaustives, comparaisons de flottants, mauvais nommage).
- ✂ AUTO-REC1 — Tracer l'exécution d'une fonction récursive simple

— Vacances de Noël —

P3 · 4 janv. → 5 févr. · 5 semaines

D2 S5 · Arbres Cr · De · Ra

- Vocabulaire : racine, nœud, feuille, hauteur, taille
- Arbres binaires ; arbres binaires de recherche (ABR)
- Parcours (préfixe, infixé, suffixe, largeur) — récursifs
- Insertion et recherche dans un ABR
- Implémentations (classes / dictionnaires)
- 🎯 STR3 — Identifier des situations nécessitant une structure arborescente ; évaluer des mesures d'un arbre binaire (taille, encadrement de la hauteur).
- 🎯 ALG1 — Calculer la taille et la hauteur d'un arbre ; le parcourir (infixé, préfixe, suffixe, largeur d'abord) ; rechercher/insérer une clé dans un arbre de recherche.
- ✂ AUTO-ARBRE1 — Connaître la hauteur d'un arbre réduit à un nœud
- 🚀 TP-ARB-01 — Arbre de décision pour IA ennemie (2h, ★★★★★)
- 🚀 TP-ARB-02 — Arbre de choix narratifs (2h, ★★★)
- 🚀 TP-ARB-03 — ABR de scores ou de personnages (2h, ★★★★★)

D3 S6 · Graphes Cr · De · Ra

- Vocabulaire : sommets, arêtes, orienté / non orienté, degré
- Représentations : matrice et listes d'adjacence
- Parcours en profondeur (DFS) et en largeur (BFS)
- Recherche de chemin ; détection de cycle
- Applications (réseaux, itinéraires)
- 🎯 STR4 — Modéliser des situations sous forme de graphes ; écrire les implémentations (matrice d'adjacence, liste de successeurs/prédécesseurs) et passer d'une représentation à l'autre.
- 🎯 ALG2 — Parcourir un graphe en profondeur ou en largeur d'abord ; repérer un cycle ; chercher un chemin dans un graphe.
- ✂ AUTO-GRAPHE1 — Compter les arêtes d'un graphe complet
- 🚀 TP-GRA-01 — Carte de donjon sous forme de graphe (2h, ★★★)
- 🚀 TP-GRA-02 — BFS/DFS pour trouver un chemin (2h, ★★★★★)
- 🚀 TP-GRA-03 — Réseau de personnages / alliances / équipes (1h30, ★★★)

— Vacances de l'Hiver —

03 Éval ECE #4 — simulation & classes avancées Cr · Ra

- Évaluation individuelle chronométrée, sujet unique adapté au niveau (POO + état qui évolue dans le temps).
- Sujet 7 — coccinelles/pucerons (simulation, classe avec méthodes d'évolution) :
https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data2026/26_BCG_NSI_7/sujet.pdf
- Sujet 9 — objets 3D (classes Sommet/Face/Objet3D, transformation d'échelle) :
https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data2026/26_BCG_NSI_9/sujet.pdf
- Sujet 12 — refuge de renards (POO + import CSV, conversion de types) :
https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data2026/26_BCG_NSI_12/sujet.pdf
- Sujet 14 — évacuation d'une pièce (grille, POO, plus proche sortie) :
https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data2026/26_BCG_NSI_14/sujet.pdf
- Sujet 21 — cartes de mémorisation façon flashcards (POO, dates, répétition espacée) :
https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data2026/26_BCG_NSI_21/sujet.pdf
- 🚀 TP-POO-03 — Cartes Magic générées depuis des objets Python (2h, ★★★)

R1 S7 · Réseaux & sécurité De · En · Ra

- Protocoles, encapsulation, modèle en couches
- Routage (RIP, OSPF — principes)
- Adressage IP, bits / paquets
- Sécurité : chiffrement symétrique / asymétrique (principes)
- Enjeux : confidentialité, intégrité, authentification
- 🎯 RES1 — Identifier, suivant le protocole de routage utilisé (RIP : nombre de sauts, OSPF : coût des routes), la route empruntée par un paquet.
- 🎯 RES2 — Décrire les principes de chiffrement symétrique et asymétrique ; décrire l'échange d'une clef symétrique via un protocole asymétrique pour sécuriser une communication HTTPS.
- ✂ AUTO-RESEAU1 — Connaître la taille d'une adresse IPv4
- 🚀 TP-RES-01 — Routage d'un paquet dans un réseau de jeu (1h30, ★★★)
- 🚀 TP-RES-02 — Chiffrement simple de messages entre joueurs (1h30, ★★★)
- 🚀 TP-RES-03 — HTTPS expliqué par échange de clés (1h, ★★★)

S1 S8 · Architectures matérielles & systèmes De · En

- Architectures (von Neumann, multicœur)
- Système d'exploitation : rôle, ressources
- Gestion des processus ; interblocage (notion)
- Commandes système ; gestion de la mémoire
- Calcul parallèle (sensibilisation)
- 🎯 ARC2 — Décrire la création d'un processus et l'ordonnancement par le système ; mettre en évidence le risque d'interblocage (deadlock).
- 🎯 ARC1 — Identifier les principaux composants d'un système sur puce (SoC) sur un schéma de circuit et les avantages de leur intégration.

— Vacances de l'Printemps —

L1 S9 · Algorithmique (paradigmes de synthèse) Ra · Cr · De

- Diviser pour régner (tri fusion, dichotomie)
- Programmation dynamique (rendu de monnaie, plus court chemin)
- Algorithmes gloutons
- k plus proches voisins (kNN)
- Synthèse algorithmes sur arbres / graphes et coûts
- Correction, terminaison, complexité
- 🌀 ALG3 — Écrire un algorithme utilisant la méthode « diviser pour régner » (ex. rotation d'image, tri fusion en $n \cdot \log_2 n$).
- 🌀 ALG4 — Utiliser la programmation dynamique pour écrire un algorithme (alignement de séquences, rendu de monnaie).
- 🌀 ALG5 — Étudier l'algorithme de Boyer-Moore pour la recherche d'un motif dans un texte ; expliquer l'intérêt du prétraitement du motif.
- 🚀 TP-ALG-T-01 — Comparer glouton, BFS et kNN dans un déplacement (2h, ★★★★★)
- 🚀 TP-ALG-T-02 — kNN pour recommander un héros ou une carte (2h, ★★★★★)
- 🚀 TP-ALG-T-03 — Analyse de coût sur une vague d'ennemis (1h30, ★★★)

L2 Éval ECE #5 — révision généraliste (mix BNS 2026) Cr · Ra · Co

- Dernière ligne droite avant les ECE (1^{re} semaine de juin) : plusieurs créneaux individuels chronométrés sur des sujets variés, pour couvrir large avant le jour J. Piocher selon le profil de chaque élève :
- Sujet 1 — compression RLE d'image (listes, bug de dépassement) :
https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data/EP_sujets0_3/EP-BAC-NSI-Sujet3.pdf
- Sujet 3 — cycle et dates (calendrier, iCalendar, formatage de chaîne) :
https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data/EP_sujets0_1/EP-BAC-NSI-Sujet1.pdf
- Sujet 8 — addition en BCD (représentation binaire, retenues) :
https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data2026/26_BCG_NSI_8/sujet.pdf
- Sujet 10 — qualité de l'eau (lissage, moyenne glissante, cas limites) :
https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data2026/26_BCG_NSI_10/sujet.pdf
- Sujet 13 — ballon-sonde (CSV, conversions, génération KML) :
https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data2026/26_BCG_NSI_13/sujet.pdf
- Sujet 16 — warming stripes (dico, graphique matplotlib) :
https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data2026/26_BCG_NSI_16/sujet.pdf
- Sujet 18 — températures Polynésie (dico, regroupement par décennie) :
https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data2026/26_BCG_NSI_18/sujet.pdf
- Sujet 20 — empreinte carbone comparée (dico, pourcentages, division par zéro) :
https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data2026/26_BCG_NSI_20/sujet.pdf
- Sujet 22 — QR code / ASCII (binaire, encodage/décodage) :
https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data2026/26_BCG_NSI_22/sujet.pdf
- Sujet 23 — trame & bits de parité (POO, contrôle d'erreur) :
https://glassus.github.io/terminale_nsi/T6_6_Epreuve_pratique/data2026/26_BCG_NSI_23/sujet.pdf